Текст задания 4

4. Задание

Предложите доработку системы для поддержки фактических отпусков.

4.1. Спроектировать таблицу для хранения фактических отпусков

Указать:

- название таблицы;
- основные поля и их типы;
- первичный ключ;
- внешние ключи;
- ограничения, которые считаете необходимыми.

Кратко пояснить принятые решения.

4.2. Спроектировать REST API для работы с фактическими отпусками

Необходимо предусмотреть:

- создание фактического отпуска;
- получение фактического отпуска;
- изменение;
- удаление;
- отправку на согласование.

Для каждого метода достаточно указать:

- HTTP-метод и endpoint;
- назначение;
- пример request body, если необходим;
- пример response body;
- основные проверки;
- основные ошибки.

4.3. Описать алгоритм создания фактического отпуска на основании планового

Необходимо описать последовательность действий системы от получения запроса до сохранения фактической заявки.

4.4. Ответить на вопросы

- Что должна сделать система, если два запроса на создание фактического отпуска по одной и той же плановой заявке поступили практически одновременно?
- Как бы вы обеспечили выполнение требования «одна плановая заявка может иметь не более одной фактической заявки»?
- Какие дополнительные вопросы вы бы задали заказчику перед началом разработки?

4.1 База данных

Задание 4.1. Спроектировать таблицу для хранения фактических отпусков

Указать:

- название таблицы;
- основные поля и их типы;
- первичный ключ;
- внешние ключи;
- ограничения, которые считаете необходимыми.

Кратко пояснить принятые решения.

Решение:

1. Информация о фактических отпусках хранится в таблице **fact_vacation_request**

2. Основные поля и их типы

Поле	**Ключ**	**Тип**	**Описание**	**Ограничения**
id	PK	UUID	Идентификатор фактической заявки	DEFAULT
planned_vacation_request_id	FK - vacation_request.id	UUID	Идентификатор плановой заявки	UNIQUE, NULL
employee_id	FK - employee.id	UUID	Идентификатор сотрудника	NOT NULL
vacation_type_id	FK - vacation_type.id	UUID	Вид отпуска	NOT NULL
date_from		DATE	Дата начала отпуска	NOT NULL

date_to		DATE	Дата окончания отпуска	NOT NULL, CHECK (date_from <= date_to)
status		VARCHAR	Статус заявки	NOT NULL, DEFAULT 'DRAFT'
description		TEXT	Дополнительная информация о заявке	NULL
approved_by	FK - employee.id	UUID	Идентификатор согласующего сотрудника	NULL
approved_at		TIMESTAMP	Дата согласования	NULL
rejection_reason		TEXT	Причина отказа	NULL
created_at		TIMESTAMP	Дата создания	NOT NULL, DEFAULT CURRENT_TIMESTAMP
updated_at		TIMESTAMP	Дата изменения	NOT NULL, DEFAULT CURRENT_TIMESTAMP

PK - первичный ключ

FK - внешний ключ

UNIQUE - уникальное значение

NULL / NOT NULL - обязательность поля

3. Первичный ключ

id - идентификатор фактической заявки

4. Внешние ключи

planned_vacation_request_id - vacation_request.id

employee_id - employee.id

vacation_type_id - vacation_type.id

approved_by - employee.id

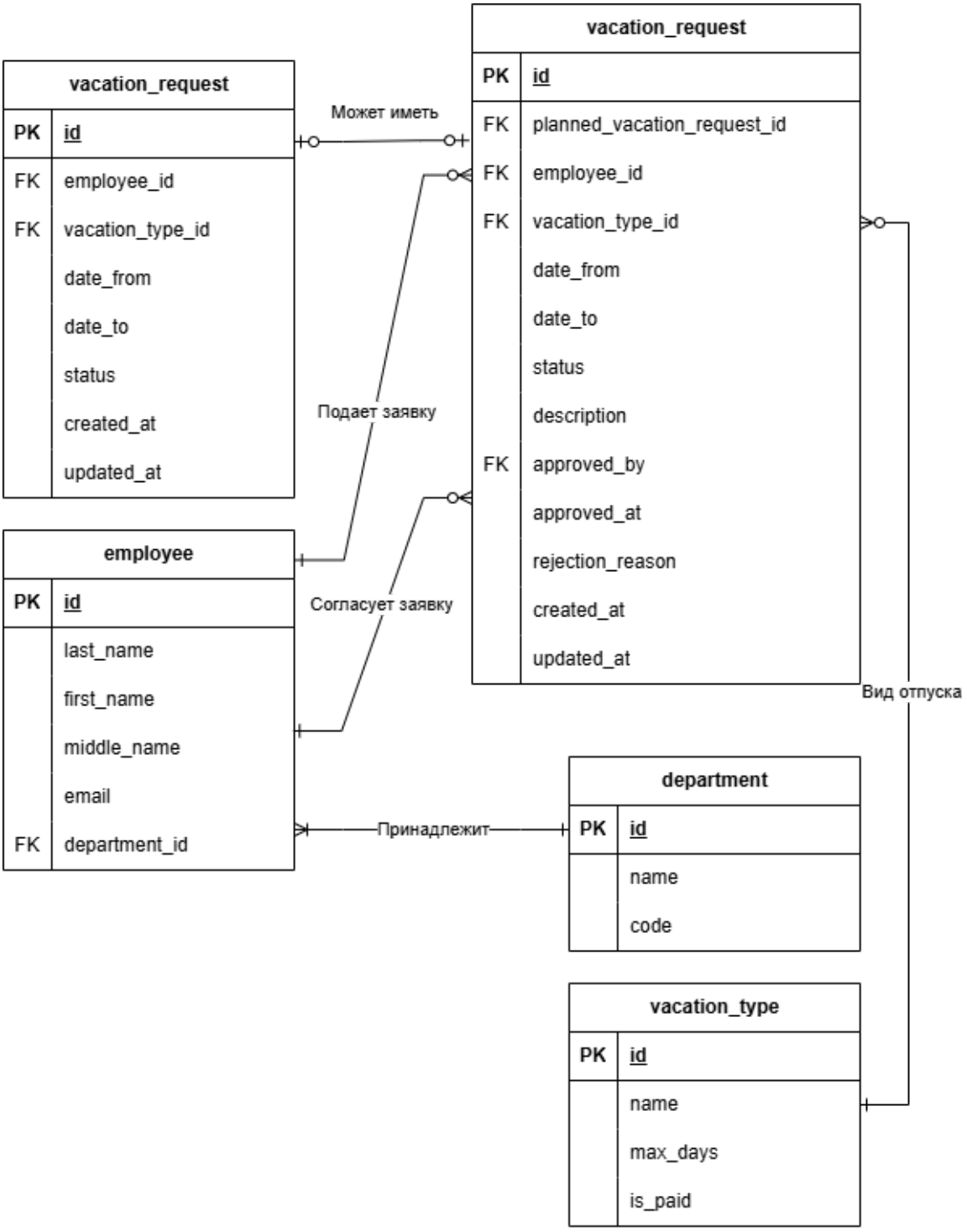
5. Ограничения

1. UNIQUE на planned_vacation_request_id гарантирует, что одна плановая заявка может быть связана не более чем с одной фактической
2. CHECK (date_from <= date_to) - дата окончания не может быть раньше даты начала
3. DEFAULT 'DRAFT' для status - при создании заявка автоматически получает статус черновика
4. DEFAULT CURRENT_TIMESTAMP для created_at и updated_at - автоматическое проставление дат

6. Индексы

- INDEX (employee_id, status) для быстрого поиска по сотруднику и статусу
- INDEX (planned_vacation_request_id) для поиска по связи с плановой заявкой

7. ER-диаграмма



8. Пояснение принятых решений

1. planned_vacation_request_id = NULL поле опционально, так как фактический отпуск может быть создан как из планового, так и самостоятельно.

2. UNIQUE на planned_vacation_request_id реализует требование "одна плановая заявка - не более одной фактической".

3. approved_by и approved_at - вынесены отдельно, чтобы хранить информацию о том, кто и когда утвердил/отклонил заявку. Поле может быть NULL до момента принятия решения.

4. employee_id, vacation_type_id, date_from, date_to обязательны для фактической заявки, так как каждая фактическая заявка должна принадлежать конкретному сотруднику, иметь вид отпуска и даты.

5. CHECK на даты нужен как защита от невалидных данных на уровне базы данных.

6. Индексы добавлены для оптимизации производительности, так как в компании 3000+ сотрудников и запросы по сотруднику и статусу будут наиболее частыми

Дополнение. Для отслеживания изменений статусов и других полей используется отдельная таблица аудита fact_vacation_request_audit, в которую записываются все изменения с указанием автора и времени. Детальное проектирование таблицы аудита выходит за рамки текущей задачи, поэтому подробно не описывается, но учитывать логирование изменений важно.

4.2 REST API

4.2. Спроектировать REST API для работы с фактическими отпусками

Необходимо предусмотреть:

- создание фактического отпуска;
- получение фактического отпуска;
- изменение;
- удаление;
- отправку на согласование.

Для каждого метода достаточно указать:

- HTTP-метод и endpoint;
- назначение;
- пример request body, если необходим;
- пример response body;
- основные проверки;
- основные ошибки.

Решение:

1. POST /fact-vacation-requests

Назначение - создание фактической заявки на отпуск.

Поддерживает два сценария:

- с привязкой к плановой заявке, если передан plannedVacationRequestId);
- самостоятельное создание.

При создании автоматически присваивается статус DRAFT.

Пример request body


```
{
  "plannedVacationRequestId": "550e8400-e29b-41d4-a716-446655440000",
```

```
  "employeeId": "550e8400-e29b-41d4-a716-446655440001",
  "vacationTypeId": "550e8400-e29b-41d4-a716-446655440002",
  "dateFrom": "2026-08-01",
  "dateTo": "2026-08-14",
  "description": "Ежегодный оплачиваемый отпуск"
}
```


Пример response body (201 Created)


```
{
  "id": "550e8400-e29b-41d4-a716-446655440003",
  "plannedVacationRequestId": "550e8400-e29b-41d4-a716-446655440000",
  "employeeId": "550e8400-e29b-41d4-a716-446655440001",
  "vacationTypeId": "550e8400-e29b-41d4-a716-446655440002",
  "dateFrom": "2026-08-01",
  "dateTo": "2026-08-14",
  "status": "DRAFT",
  "description": "Ежегодный оплачиваемый отпуск",
  "approvedBy": null,
  "approvedAt": null,
  "rejectionReason": null,
  "createdAt": "2026-07-23T10:30:00Z",
  "updatedAt": "2026-07-23T10:30:00Z"
}
```


Проверки

- Проверяется наличие обязательных полей: employeeId, vacationTypeId, dateFrom, dateTo.
- Дата начала не может быть позже даты окончания.
- Проверяется существование сотрудника и вида отпуска.
- Если передан plannedVacationRequestId: проверяется существование плановой заявки, её статус = PLANNED, а также отсутствие связанной фактической заявки.

Ошибки

HTTP статус	Код ошибки	Описание
400	INVALID_DATE_FORMAT	Неверный формат даты
404	EMPLOYEE_NOT_FOUND	Сотрудник не найден
404	VACATION_TYPE_NOT_FOUND	Вид отпуска не найден
404	PLANNED_REQUEST_NOT_FOUND	Плановая заявка не найдена
409	PLANNED_REQUEST_NOT_PLANNED	Плановая заявка не в статусе PLANNED
409	FACT_REQUEST_ALREADY_EXISTS	У плановой заявки уже есть фактическая

Пример ошибки 409

При попытке создать фактическую заявку из плановой, которая не находится в статусе PLANNED

{

"code": "PLANNED_REQUEST_NOT_PLANNED",

"message": "Фактический отпуск можно создать только из заявки в статусе PLANNED. Текущий статус: PENDING"

}

---

2. GET /fact-vacation-requests/{id}

Назначение - получение информации о конкретной фактической заявке по её идентификатору.

Пример response body (200 OK)

{

"id": "550e8400-e29b-41d4-a716-446655440003",

"plannedVacationRequestId": "550e8400-e29b-41d4-a716-446655440000",

"employeeId": "550e8400-e29b-41d4-a716-446655440001",

"vacationTypeId": "550e8400-e29b-41d4-a716-446655440002",

"dateFrom": "2026-08-01",

"dateTo": "2026-08-14",

"status": "DRAFT",

"description": "Ежегодный оплачиваемый отпуск",

"approvedBy": null,

"approvedAt": null,

"rejectionReason": null,

"createdAt": "2026-07-23T10:30:00Z",

"updatedAt": "2026-07-23T10:30:00Z"

}

Проверки

- заявка с указанным id существует;
- у пользователя есть права на просмотр.

Ошибки

HTTP статус	Код ошибки	Описание
403	ACCESS_DENIED	Недостаточно прав
404	FACT_REQUEST_NOT_FOUND	Заявка не найдена

Пример ошибки 404

{

"code": "FACT_REQUEST_NOT_FOUND",

"message": "Заявка с идентификатором 550e8400-e29b-41d4-a716-446655440003 не найдена"

}

---

3. PATCH /fact-vacation-requests/{id}

Назначение - частичное обновление фактической заявки. Доступно только в статусе DRAFT.

Передаются только изменяемые поля: vacationTypeId, dateFrom, dateTo, description. Поля employeeId и plannedVacationRequestId изменять нельзя, так как это нарушит целостность данных.

Пример request body


```
{
  "vacationTypeId": "550e8400-e29b-41d4-a716-446655440005",
  "dateFrom": "2026-08-15",
  "dateTo": "2026-08-28",
  "description": "Изменен вид отпуска и даты"
}
```


Пример response body (200 OK)


```
{
  "id": "550e8400-e29b-41d4-a716-446655440003",
  "plannedVacationRequestId": "550e8400-e29b-41d4-a716-446655440000",
  "employeeId": "550e8400-e29b-41d4-a716-446655440001",
  "vacationTypeId": "550e8400-e29b-41d4-a716-446655440005",
  "dateFrom": "2026-08-15",
  "dateTo": "2026-08-28",
  "status": "DRAFT",
  "description": "Изменен вид отпуска и даты",
  "approvedBy": null,
  "approvedAt": null,
  "rejectionReason": null,
  "createdAt": "2026-07-23T10:30:00Z",
  "updatedAt": "2026-07-23T11:45:00Z"
```

}

Проверки

- заявка существует;
- статус = DRAFT;
- даты корректны, выполняется dateFrom <= dateTo;
- если передан vacationTypeId, то он существует;
- у пользователя есть права на изменение.

Ошибки

HTTP статус	Код ошибки	Описание
400	INVALID_DATE_FORMAT	Неверный формат даты
403	ACCESS_DENIED	Недостаточно прав
404	FACT_REQUEST_NOT_FOUND	Заявка не найдена
409	INVALID_STATUS_FOR_UPDATE	Изменение доступно только в статусе DRAFT

Пример ошибки 409

{

"code": "INVALID_STATUS_FOR_UPDATE",

"message": "Изменение заявки возможно только в статусе DRAFT. Текущий статус: PENDING"

}

---

4. DELETE /fact-vacation-requests/{id}

Назначение - удаление фактической заявки.

Доступно только в статусе DRAFT. Если заявка была создана из плановой, связь разрывается.

Проверки

- заявка существует;
- статус = DRAFT;

- у пользователя есть права на удаление.

Ошибки

HTTP статус	Код ошибки	Описание
403	ACCESS_DENIED	Недостаточно прав
404	FACT_REQUEST_NOT_FOUND	Заявка не найдена
409	INVALID_STATUS_FOR_DELETE	Удаление доступно только в статусе DRAFT

Пример ошибки 409

{

"code": "INVALID_STATUS_FOR_DELETE",

"message": "Удаление заявки возможно только в статусе DRAFT. Текущий статус: PENDING"

}

---

5. POST /fact-vacation-requests/{id}/submit

Назначение - отправка заявки на согласование. Статус меняется с DRAFT на PENDING, обновляется updated_at.

Пример response body (200 OK)

{

"id": "550e8400-e29b-41d4-a716-446655440003",

"plannedVacationRequestId": "550e8400-e29b-41d4-a716-446655440000",

"employeeId": "550e8400-e29b-41d4-a716-446655440001",

"vacationTypeId": "550e8400-e29b-41d4-a716-446655440005",

"dateFrom": "2026-08-15",

"dateTo": "2026-08-28",

"status": "PENDING",

"description": "Ежегодный оплачиваемый отпуск",

"approvedBy": null,

"approvedAt": null,

"rejectionReason": null,

"createdAt": "2026-07-23T10:30:00Z",

"updatedAt": "2026-07-23T12:00:00Z"

}

Проверки

- заявка существует;
- статус = DRAFT;
- у пользователя есть права.

Ошибки

HTTP статус	Код ошибки	Описание
403	ACCESS_DENIED	Недостаточно прав
404	FACT_REQUEST_NOT_FOUND	Заявка не найдена
409	INVALID_STATUS_FOR_SUBMIT	Отправка доступна только в статусе DRAFT

Пример ошибки 409

{

"code": "INVALID_STATUS_FOR_SUBMIT",

"message": "Отправка на согласование возможна только в статусе DRAFT. Текущий статус: PENDING"

}

4.3. Алгоритм создания

4.3. Описать алгоритм создания фактического отпуска на основании планового

Необходимо описать последовательность действий системы от получения запроса до сохранения фактической заявки.

Решение:

Создание фактической заявки может происходить по двум сценариям

- На основании плановой заявки
- Самостоятельное создание

В обоих случаях используется метод POST /fact-vacation-requests.

Сценарий 1. Создание фактической заявки из плановой заявки

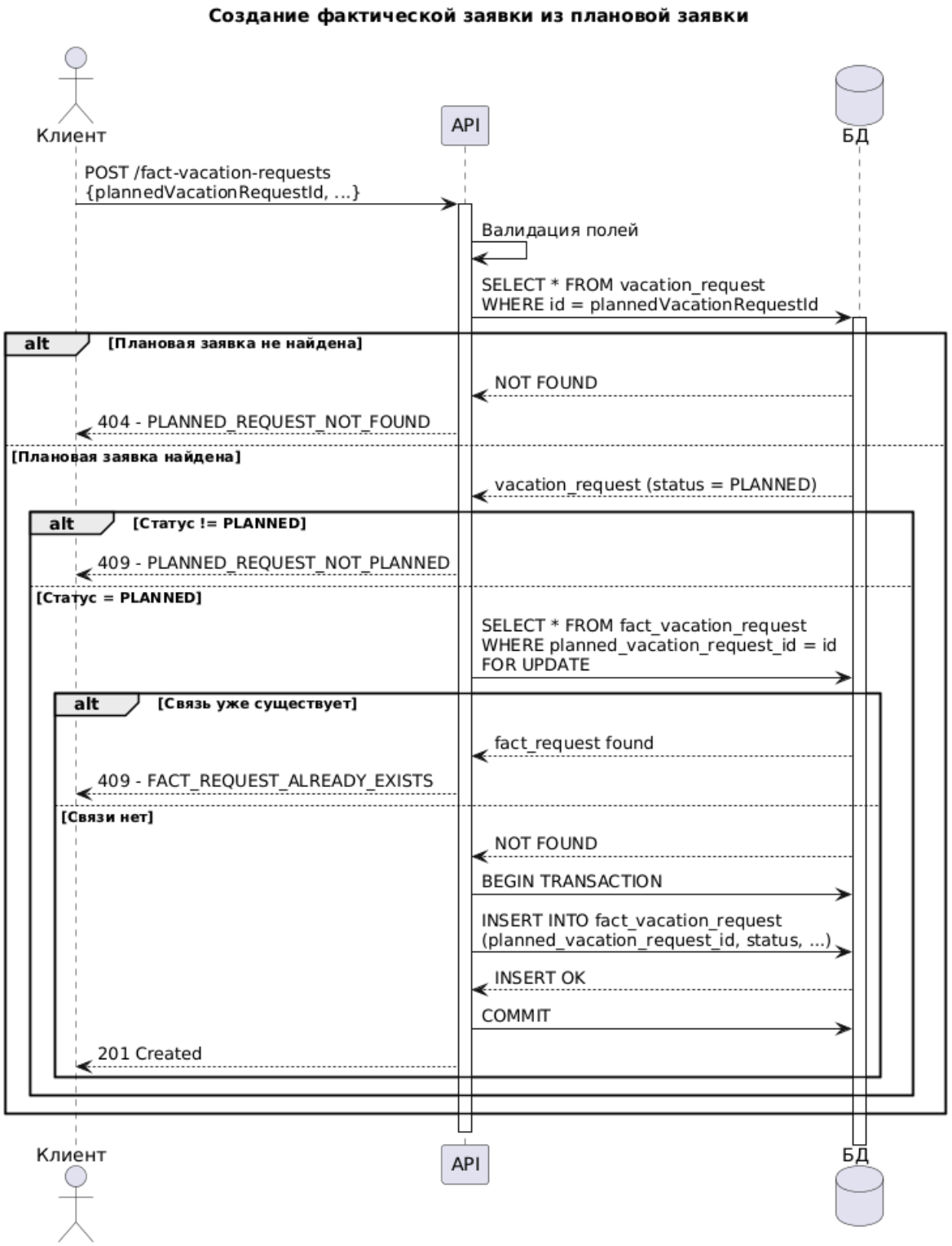
Шаг	Действие	Описание
1	Получение запроса	API принимает POST /fact-vacation-requests с телом запроса
2	Валидация обязательных полей	Проверка наличия employeeId, vacationTypeId, dateFrom, dateTo
3	Валидация дат	Проверка dateFrom <= dateTo
4	Поиск плановой заявки	SELECT * FROM vacation_request WHERE id = plannedVacationRequestId
5	Проверка существования	Если заявка не найдена, то ошибка 404
6	Проверка статуса	Если статус != PLANNED, то ошибка 409
7	Поиск уникальности связи в БД	SELECT * FROM fact_vacation_request WHERE planned_vacation_request_id = plannedVacationRequestId

8	Результат проверки на дублирование	Если запись найдена, то ошибка 409
9	Сохранение	Открытие транзакции, INSERT, фиксация транзакции
10	Формирование ответа	Возврат 201 Created

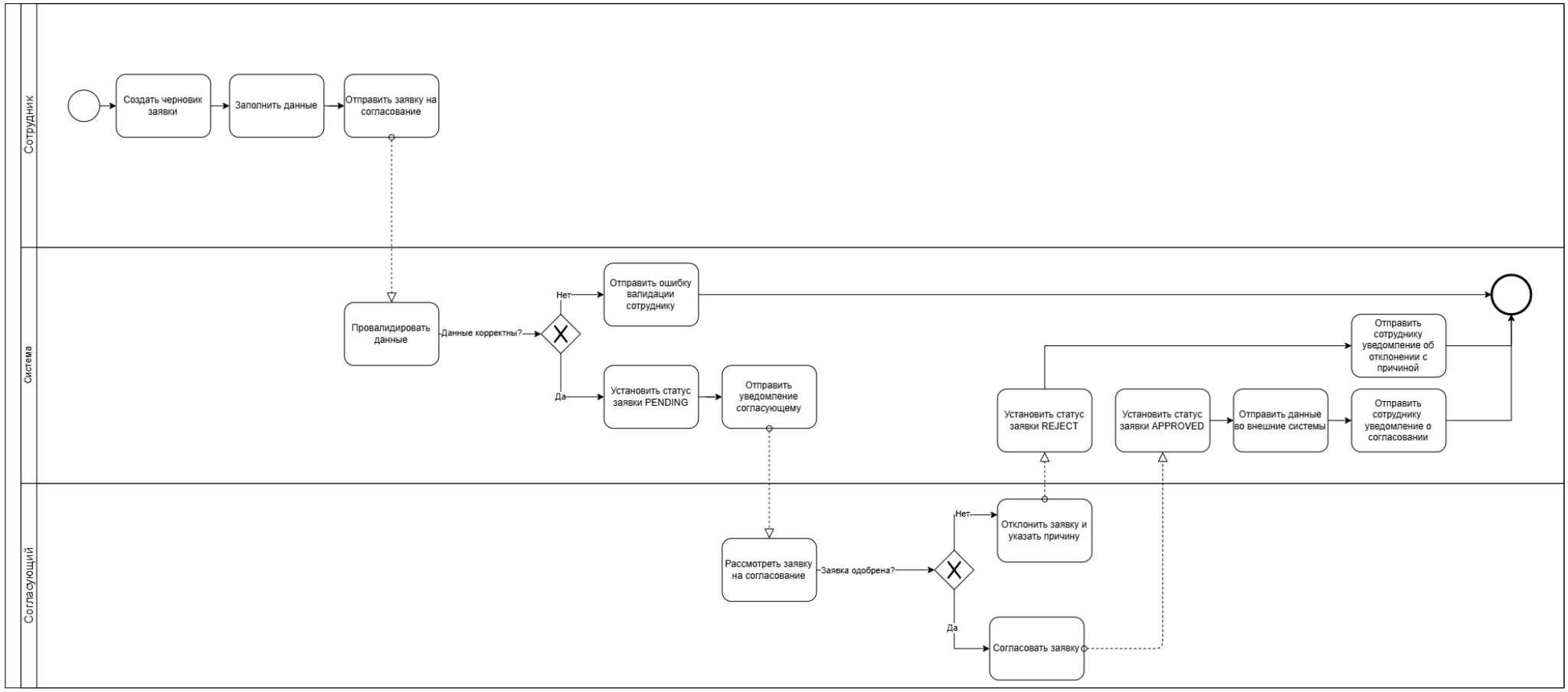
Сценарий 2. Самостоятельное создание фактической заявки

Шаг	Действие	Описание
1	Получение запроса	API принимает POST /fact-vacation-requests
2	Валидация обязательных полей	Проверка наличия employeeId, vacationTypeId, dateFrom, dateTo
3	Валидация дат	Проверка dateFrom <= dateTo. Если нет, то ошибка 400
4	Проверка существования сотрудника	SELECT * FROM employee WHERE id = employeeId. Если нет, то ошибка 404
5	Проверка существования вида отпуска	SELECT * FROM vacation_type WHERE id = vacationTypeId. Если нет, то ошибка 404
6	Сохранение	Открытие транзакции, INSERT, фиксация транзакции
7	Формирование ответа	Возврат 201 Created с полным объектом заявки

Sequence Diagram для Сценария 1. Создание фактической заявки из плановой заявки



BPMN-диаграмма процесса создания и согласования фактической заявки на отпуск



4.4 Ответ на вопросы

4.4. Ответить на вопросы

- Что должна сделать система, если два запроса на создание фактического отпуска по одной и той же плановой заявке поступили практически одновременно?
- Как бы вы обеспечили выполнение требования «одна плановая заявка может иметь не более одной фактической заявки»?
- Какие дополнительные вопросы вы бы задали заказчику перед началом разработки?

Решение:

Вопрос 1. Что должна сделать система, если два запроса на создание фактического отпуска по одной и той же плановой заявке поступили практически одновременно?

При одновременном поступлении двух запросов система должна гарантировать, что только один запрос будет успешно обработан. Второй запрос получит ошибку 409 Conflict с кодом FACT_REQUEST_ALREADY_EXISTS.

Способы реализации:

1. Блокировка на уровне БД. При проверке плановой заявки используется SELECT ... FOR UPDATE. Это блокирует строку на время обработки запроса. Второй запрос ждет, пока первый завершится, затем проверяет наличие связи и получает ошибку. Все операции выполняются в рамках одной транзакции. Перед созданием также проверяется, что плановая заявка всё ещё в статусе PLANNED.
2. Дополнительная страховка с помощью ограничения UNIQUE на поле planned_vacation_request_id в таблице fact_vacation_request. Даже если оба запроса пройдут проверки одновременно, вставка второй записи вызовет ошибку дублирования.
3. Альтернативно можно рассмотерть очередь сообщений. Запросы помещаются в очередь и обрабатываются последовательно.

Вопрос 2. Как бы вы обеспечили выполнение требования «одна плановая заявка может иметь не более одной фактической заявки»?

1. На уровне бэкенда делать проверку перед созданием заявки. Перед созданием фактической заявки система выполняет проверку:

   SELECT * FROM fact_vacation_request WHERE planned_vacation_request_id = ?

   Если запись найдена, возвращается ошибка.

2. На уровне БД на поле planned_vacation_request_id в таблице fact_vacation_request наложено ограничение UNIQUE. База данных физически не позволит вставить вторую запись с таким же planned_vacation_request_id.

Бэкенд возвращает понятную пользователю ошибку. База данных - это последняя линия защиты, которая гарантирует уникальность даже при одновременных запросах или ошибках в бэкенде.

Вопрос 3. Какие дополнительные вопросы вы бы задали заказчику перед началом разработки?

1. Для чего в целом нужны фактические отпуска? На что они влияют и где будут использоваться?
2. Какая ожидаемая нагрузка? Сколько заявок в день/месяц?
3. Есть ли требования по времени ответа API?
4. Нужно ли отправлять уведомления сотрудникам и руководителям при создании или утверждении заявки?
5. Кто может создавать фактические заявки?
6. Кто может изменять и удалять заявку в статусе DRAFT?
7. Кто утверждает заявки?
8. В какие системы передаются данные об утвержденном отпуске?
9. Нужно ли собирать статистику отпусков? Если да, то для чего и что там должно храниться?
10. Нужно ли хранить полную историю изменений заявки? Какие данные обязательно должны быть?